SCHEMA-DRIVEN BACKEND GENERATOR FOR AUTOMATED DTO AND OPENAPI SPECIFICATION CREATION

25-26J-436

Project Proposal Report

Thamindu Vimansha Dissanayake

B.Sc. (Hons) in Information Technology, Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

SCHEMA-DRIVEN BACKEND GENERATOR FOR AUTOMATED DTO AND OPENAPI SPECIFICATION CREATION

25-26J-436

Project Proposal Report

B.Sc. (Hons) in Information Technology, Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

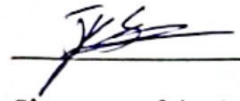
DECLARATION

Declaration page of the candidates & supervisor

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Dissanayake D M T V	IT22003546	

The above candidate is carrying out research for the undergraduate dissertation under my supervision.



Signature of the Supervisor

27/08/2025

Date

Signature of the Co-Supervisor

28/08/2025

Date

ACKNOWLEDGEMENT

Working on this final-year research project has been an immensely rewarding academic experience. Throughout the process of preparing the research proposal, I have gained valuable knowledge about the research process, including identifying research problems, structuring methodologies, and recognizing potential challenges.

I would like to extend my sincere gratitude to my supervisor, Mr. Jeewaka Perera, for his invaluable guidance, constructive feedback, and continuous encouragement throughout this project. His insightful suggestions and support have been instrumental in shaping this work. I am equally grateful to my co-supervisor, Ms. Thilini Jayalath, for her dedicated guidance, thoughtful feedback, and encouragement, which have greatly enriched the quality of this research. I also wish to express my appreciation to all the lecturers at Sri Lanka Institute of Technology for their support and the strong academic foundation they have provided.

I look forward to building on this experience, further strengthening my research skills, and contributing meaningful insights through the successful completion of this project.

ABSTRACT

The increasing demand for cross-platform applications has highlighted the importance of schema-driven automation in reducing development effort and maintaining consistency across system layers. The research project AI Powered Cross-Platform Form Generator with Schema Driven Full-Stack Automation is designed to address this need by transforming high-level schema definitions into backend and frontend artifacts. Within this system, the backend generator plays a critical role by converting JSON Schema inputs into Data Transfer Objects (DTOs) with validation annotations and generating OpenAPI specifications for interoperability.

A review of related work shows that while existing research and industrial tools address parts of the problem, such as REST API auto-generation [1], OpenAPI specification extraction [2], validation frameworks [4], and server stub generation through Swagger and OpenAPI Generator [5][6][7], they remain fragmented. Current solutions often produce either service skeletons or specifications, but do not integrate schema-to-backend generation, validation embedding, and specification synthesis into a unified process. Security [9] further demonstrate the importance of correctness in generated code, yet these are applied post-generation rather than embedded within the pipeline.

The proposed backend generator addresses this gap by employing a schema-driven methodology that directly interprets JSON Schema definitions to derive validation rules and constraints. This ensures that backend outputs remain aligned with the original schema specifications. The component generates platform-ready DTOs together with synchronized OpenAPI model specifications, enabling seamless integration with frontend generators and middleware for full-stack automation. This contribution improves consistency across system layers, minimizes manual coding effort, and enhances interoperability, while also providing a foundation for future extensions in areas such as security and compliance.

Keywords: *Schema-driven automation, Backend generation, JSON Schema, Data Transfer Objects (DTOs), Validation annotations, OpenAPI specification, Cross-platform form generator, Full-stack automation, Interoperability.*

Table Of Contents

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

The increasing reliance on schema-driven development has transformed how modern software systems are designed and deployed. Enterprises are required to deliver applications across multiple platforms such as web, mobile, and cloud environments, where uniformity and maintainability are essential. Traditional development methods demand significant manual effort to ensure that frontend interfaces, backend services, and integration points remain consistent, often resulting in duplicated logic and higher risks of inconsistencies. To address these challenges, the research project AI-Powered Cross-Platform Form Generator with Schema-Driven Full-Stack Automation introduces a system that can automatically generate application layers from high-level specifications [1].

The system as a whole is organized into several interconnected components that work together to transform schema definitions into functioning software artifacts. Among these, the backend generator and security integration component forms the foundation of the service layer. This component accepts JSON schema as its input, from which it generates Spring Boot Data Transfer Objects (DTOs) with validation annotations and produces OpenAPI specifications. These generated artifacts enable seamless integration with other system modules while ensuring that the backend remains aligned with schema definitions [2].

Several studies emphasize the difficulties faced when attempting to manually synchronize backend logic with schema definitions. Inconsistencies between client-side validations and server-side checks have been widely documented, often leading to failures in data integrity and increased maintenance effort [3]. Research on automatic API specification generation further highlights the importance of accurate and machine-readable service descriptions such as OpenAPI, which provide a standardized way to document and consume backend services across heterogeneous environments [4]. While these approaches address specific aspects of automation, they typically do not provide a unified mechanism for translating schemas directly into validated backend components and formal API specifications.

In this context, the backend generator component of the proposed system makes a focused contribution by bridging this gap. By transforming JSON schema into DTOs with embedded validation rules and automatically creating OpenAPI specifications, the component ensures that backend artifacts are generated in a consistent and verifiable manner. This not only supports the larger goal of schema-driven full-stack automation but also enhances interoperability and reduces the dependency on manual coding. Consequently, the backend generator serves as a key element in enabling the system to produce reliable and maintainable applications across platforms.

1.1 Background

The demand for applications that run across different platforms has increased as organizations look for ways to reduce development time while keeping consistency in design and function. In traditional software projects, developers often write separate backend services, data transfer objects, and validation logic for each application layer. This leads to duplication of work and a higher chance of mismatches between the frontend and backend. Studies point out that even small inconsistencies in validation rules or data handling can create reliability problems and raise maintenance costs [5]. These challenges have encouraged research into schema-driven automation where the schema becomes the main source of truth for generating both backend and frontend components.

The system proposed under this project, AI Powered Cross-Platform Form Generator with Schema Driven Full-Stack Automation, follows this schema-driven approach. It is designed to receive structured definitions, from which different components are generated automatically. The frontend generator focuses on building user interfaces, while the backend generator is responsible for producing backend artifacts such as DTOs and API specifications. Together, these parts aim to cut down the manual work involved in ensuring that forms and data validations are consistent across multiple platforms [1].

The backend generator component specifically takes a JSON schema as input. JSON schema provides a widely used standard for defining the structure, constraints, and

validation requirements of data objects. When used as the input to the backend generator, the schema guides the creation of DTOs with validation annotations in Spring Boot. This ensures that the same validation logic defined in the schema is enforced in the backend, reducing the likelihood of data mismatches between the client and server. Alongside DTO generation, the component also produces OpenAPI specifications which describe the endpoints and data models in a machine-readable format. OpenAPI has become a common choice in industry for documenting RESTful services and supporting integration between independent systems [6][7].

Previous research shows the importance of combining schema-driven design with automated code generation. Approaches such as model-based API generation and transformation-driven specification learning have already demonstrated that automation can improve development productivity and lower human error [6][8]. However, these studies often concentrate on one part of the backend, such as generating APIs or DTOs, without addressing the full pipeline from schema to validated backend artifacts and service specifications. By filling this space, the backend generator in the proposed system contributes not only to the automation of backend code but also to the consistency of the entire system.

1.2 Literature Survey

Schema-driven code generation has been identified as a key strategy for reducing development effort and improving consistency in backend systems. Approaches that automatically generate REST APIs from abstract definitions show that automation can improve productivity and reduce the burden on developers [1]. Other research demonstrates that transformation-based methods can reconstruct OpenAPI specifications from unstructured documentation with high accuracy, confirming that specification-driven generation is technically feasible [2]. These works highlight the strengths of automation but remain focused on either endpoint generation or specification extraction, without integrating validation or data transfer objects directly into backend code.

Consistency between frontend and backend has long been a challenge in client–server applications. Studies on formal consistency evaluation confirm that mismatches in

validation rules across system layers frequently cause data integrity failures and maintenance issues [3]. Comparative analyses of validation strategies indicate that annotation-based techniques are simpler to apply but less flexible, while more advanced approaches introduce complexity and overhead [4]. These findings reinforce the importance of embedding validation logic directly from schemas into backend components, which ensures that application rules are not duplicated or fragmented across layers.

The automated generation of service specifications is another major area of development. Research shows that OpenAPI documentation can be leveraged not only for backend specification but also for generating user-facing web frontends [5]. The widespread adoption of tools such as Swagger Codegen [6] and OpenAPI Generator [7] demonstrates the central role of specification-centered development in industry, where a single machine-readable contract can be used to generate client SDKs, server stubs, and documentation. These practices confirm that aligning backend generation with OpenAPI outputs is essential for interoperability and for enabling integration with widely used toolchains.

In addition, compliance frameworks such as GDPR consent management and verification tools provide evidence that backend systems increasingly need to account for regulatory requirements [11]. Although not within the immediate scope of the backend generator, these perspectives confirm that integration of compliance features is an emerging expectation for backend automation tools.

Enterprise system research emphasizes the need for interoperability across diverse services. Middleware architectures that provide RESTful access to legacy SOAP services allow organizations to preserve existing functionality while exposing lightweight modern interfaces [12]. Broader surveys of microservice integration strategies confirm that flexible, loosely coupled service composition is critical for scalability and maintainability [13]. These directions align with the backend generator's role in supporting integration by producing outputs that can be easily consumed by other system components.

Emerging studies show that intelligent automation is shaping the next generation of software engineering. Transformer-based models have demonstrated the ability to generate code directly from natural language prompts, reducing the need for repetitive coding [14]. Other research explores reinforcement learning and recommender systems for automated penetration testing, highlighting how automation extends beyond development to verification and security [15]. Together, these works reflect a shift toward full lifecycle automation, from code generation to validation and testing, which supports the overall direction of schema-driven full-stack automation.

In summary, the literature establishes that code generation frameworks [1][2], consistency evaluations [3][4], specification-centered approaches [5][6][7], and security-focused frameworks [8][9] provide strong foundations, while research on authenticity, compliance, and integration [10][11][12][13] situates backend generation within a broader system environment. Intelligent automation [14][15] points to future directions where backend generation is part of a larger movement toward complete lifecycle automation. Despite these advances, there remains a lack of an integrated pipeline that directly consumes JSON Schema and produces both DTOs with validation annotations and OpenAPI specifications. The backend generator component in the proposed system is designed to address this gap, ensuring consistency, interoperability, and alignment with the larger objective of schema-driven full-stack automation.

1.3 Research Gap

The reviewed studies make clear that automation has improved specific aspects of backend development, but no single approach provides an integrated solution. Model-based frameworks demonstrate how REST services can be automatically generated from abstract definitions [1], while transformation-based methods have achieved strong accuracy in producing OpenAPI specifications from unstructured documentation [2]. These contributions illustrate that schema-driven automation is both feasible and valuable. However, they remain focused on partial tasks such as endpoint creation or specification generation, and do not extend their scope to the automatic production of data transfer objects with embedded validation.

Consistency between application layers has been identified as another critical issue. Studies on backend and frontend logic alignment show that mismatches in validation rules lead to reliability problems and maintenance overhead [3]. Analyses of validation strategies further emphasize the trade-off between simplicity and flexibility [4]. Yet, there is limited evidence of approaches that directly embed validation annotations from a shared schema into backend components. This leaves a gap in ensuring that constraints expressed at the schema level are faithfully preserved in the backend.

In terms of service specification, OpenAPI has emerged as the dominant standard, with both academic and industry efforts showing its role in enabling automation across ecosystems [5][6][7]. While these tools and studies confirm the benefits of specification-centered development, most approaches either extract specifications from existing systems or focus on using specifications to generate additional artifacts. Few attempts have been made to automatically synthesize OpenAPI specifications in direct coordination with schema-driven backend generation.

Enterprise integration studies highlight how middleware solutions help bridge SOAP-based systems with REST APIs [12] and how microservice taxonomies classify integration approaches [13]. These works reinforce the importance of interoperability, but they again stop short of linking schema-driven DTO generation, validation embedding, and specification synthesis into one process.

Taken together, the literature shows that code generation, validation consistency, and service specification have all been studied extensively but largely in isolation. There is a clear absence of a unified backend generation pipeline that consumes JSON Schema and produces both DTOs with validation annotations and OpenAPI specifications as coordinated outputs. Filling this gap is central to achieving schema-driven full-stack automation, as it ensures that the backend layer remains consistent, interoperable, and aligned with the broader system.

Feature	Consistency Eval	Validation in Springboot	OpenAPI Tools	CodingCare/ Phoenix	Proposed System
Schema-to-Backend generation	X	X	Partial	X	✓
Validation annotations directly from schema	X	(manual, not schema-driven)	X	X	✓
Automatic OpenAPI specification generation	X	X	✓	X	✓
Consistency across frontend and backend	Partial	X	X	X	✓
Security assurance in generated code	X	X	X	Partial - (post-generation)	Future extension
Integration in full-stack automation system	X	X	Partial	X	✓

Table 1.1 - Gap comparison with already existing systems and previous research papers

1.4 Research Problem

Modern application development has moved toward schema-driven approaches in order to manage complexity and achieve consistency across platforms. While research has produced notable advances in model-based API generation [1], automated specification learning [2], validation consistency analysis [3][4]. Each addresses a single part of the challenge, such as specification generation, validation strategies, but no study presents an integrated method that ties these strands together.

The absence of such integration leaves development teams with fragmented toolchains. Developers are often required to manually construct data transfer objects, attach validation rules, and separately prepare OpenAPI specifications. This duplication introduces inconsistency, increases maintenance cost, and undermines the very purpose of schema-driven development. At the same time, industry-standard practices rely heavily on OpenAPI as the central contract for interoperability [5][6][7], meaning that if DTOs and validation logic are not aligned with the specification, downstream tools such as Swagger Codegen or OpenAPI Generator cannot be used effectively.

The research problem therefore centers on the lack of a unified backend generation pipeline that accepts a JSON Schema as the single source of truth and automatically produces two coordinated outputs: DTOs with embedded validation annotations and OpenAPI specifications. Without such a process, the promise of schema-driven full-stack automation remains incomplete, as backend development still demands manual intervention to ensure consistency and interoperability. Within the broader system of an AI-powered cross-platform form generator, addressing this problem is critical for maintaining alignment across layers and enabling the system to function as a coherent whole.

2. OBJECTIVES

The project AI Powered Cross-Platform Form Generator with Schema Driven Full-Stack Automation has been designed with the aim of reducing manual effort in full-stack development and ensuring consistency across application layers. Each component of the system contributes to this objective by transforming schema definitions into concrete software artifacts. The backend generator and security integration component addresses the service layer by focusing on automated backend code and specification generation. Its objectives have been formulated based on the research problem and gaps identified in the literature.

2.1 Main Objective

To design and implement a backend generator that automatically transforms a JSON schema into Spring Boot Data Transfer Objects (DTOs) with validation annotations and generates OpenAPI specifications, thereby ensuring consistency and interoperability within the schema-driven full-stack automation system.

2.2 Specific Objectives

- To implement schema-to-DTO conversion methods that translate JSON schema definitions into Spring Boot classes with appropriate Bean Validation annotations, ensuring that backend validation aligns with schema constraints.
- To design and develop an OpenAPI specification generator that automatically produces accurate service descriptions from the generated backend components, enabling smooth integration with frontend and middleware components of the system.
- To evaluate the generated backend artifacts against sample schemas in order to confirm correctness, completeness, and alignment with schema requirements.
- To integrate the backend generator with the larger system workflow so that it functions as a core service layer within the full-stack automation platform, supporting cross-platform frontend generation and other related components.

3. METHODOLOGY

The proposed backend generation and security integration component adopts a schema-driven approach where a single JSON Schema acts as the authoritative source for data definitions. From this schema, Spring Boot DTOs enriched with JSR-303 validation annotations and synchronized OpenAPI backend contracts are automatically produced. The framework incorporates advanced validation rules such as cross-field and conditional constraints, while embedding compliance metadata for GDPR-sensitive fields and enforcing security-focused validations including input sanitization. A continuous integration–based consistency checker is further introduced to detect mismatches between frontend and backend validations, ensuring reliability, compliance, and maintainability throughout the development process.

3.1 System Overview Diagram

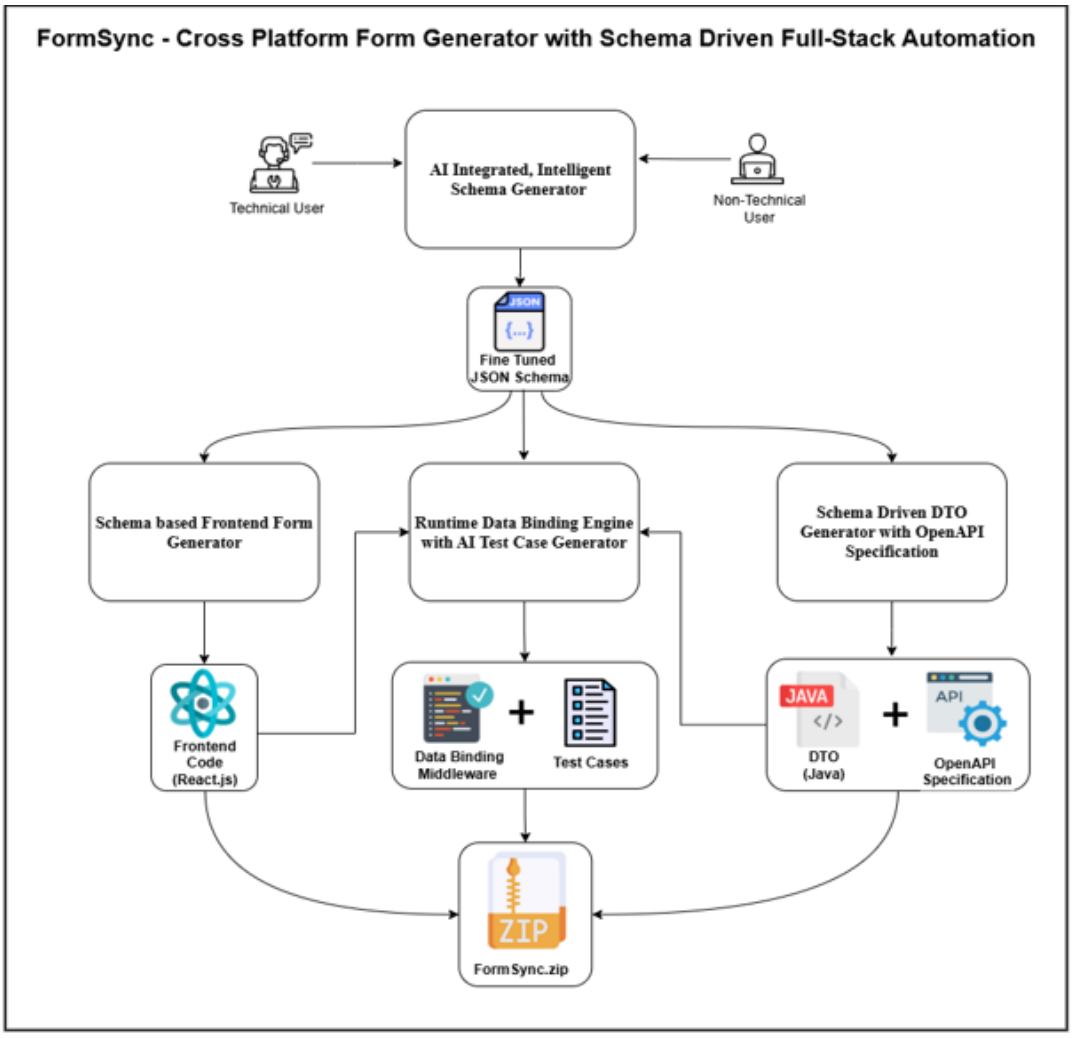


Figure 3.1 - System diagram for Cross platform form generator with schema driven automation

3.2 Component System Diagram

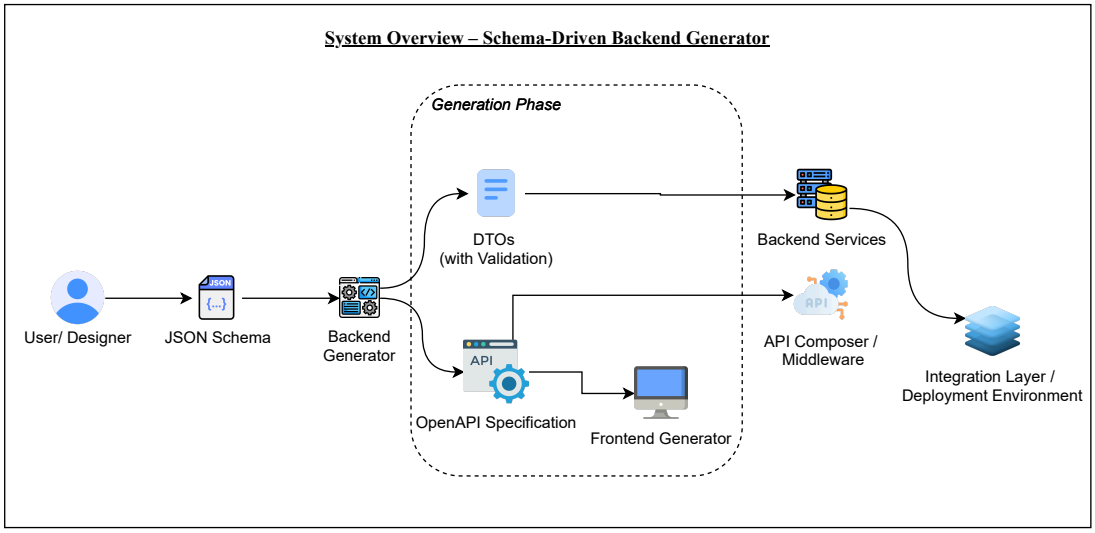


Figure 3.2 - System Diagram for Schema-Driven Backend Generator for Automated DTO and OpenAPI Specification Creation

across different layers of an application. The backend generator is developed as a standalone module within this system, responsible for producing backend artifacts that can later be integrated into the wider platform. Its role is to transform JSON Schema inputs into Spring Boot DTOs with validation annotations and to produce OpenAPI specifications that describe the generated models. By ensuring that backend artifacts are consistent with schema definitions, this component contributes directly to the larger objective of maintaining coherence between frontend, backend, and integration layers.

3.4 Component workflow

The workflow of the backend generator follows a structured sequence of stages. The process begins with schema ingestion, where a JSON Schema is provided as input. A Lexer and Parser then process the schema, breaking it into tokens and constructing a parse tree or abstract syntax tree. This stage enforces both syntactic and semantic correctness, ensuring that the schema complies with structural rules while also capturing validation constraints such as required fields, string length, numeric ranges, and patterns. By introducing a Lexer and Parser, the component guarantees that constraints are extracted accurately and can be mapped consistently into backend code and specifications.

Once the schema is parsed, an intermediate representation is created. This representation captures the properties, types, and validation rules defined in the schema in a normalized structure. It acts as the foundation for subsequent generation tasks, ensuring that both DTO creation and specification synthesis use the same underlying model.

3.5 DTO generation

The next stage is the production of Data Transfer Objects. Using the intermediate representation, DTO classes are generated in either Spring Boot, depending on the chosen output target. For Spring Boot, schema properties are mapped into Java fields annotated with standard Bean Validation annotations such as @NotNull, @Size, @Pattern, @Min, and @Max. These annotations are taken directly from the constraints extracted by the Parser, ensuring that the generated classes enforce the same rules defined in the JSON Schema. Naming conventions, package structures, and class organization follow standard practices for each platform so that the output can be immediately integrated into a service project.

3.6 OpenAPI specification synthesis

In parallel with DTO generation, the intermediate representation is also used to produce an OpenAPI specification. This specification describes the generated data models in the components.schemas section of an OpenAPI document. Each property is represented with its type, required flag, and any constraints defined in the schema. For example, string length limits are translated into minLength and maxLength, numeric ranges become minimum and maximum, and regex constraints are carried over as pattern. At this stage, the focus is on synthesizing accurate data model specifications rather than complete service definitions. These specifications will later be used by other modules of the system to compose API endpoints and connect frontend and backend layers consistently.

3.7 Evaluation approach

The evaluation of the backend generator will be conceptual rather than experimental, as the project does not implement runtime testing. The generated DTOs will be examined for correctness by checking that their fields, types, and validation

annotations align with the original schema. Similarly, the produced OpenAPI document will be validated against OpenAPI standards to ensure that the generated schemas match the input definitions. Representative JSON Schemas covering primitive types and basic entities will be used to verify that the pipeline consistently reproduces all constraints across both outputs.

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

4.1.1 User Requirements

The proposed system, AI Powered Cross-Platform Form Generator with Schema Driven Full-Stack Automation, enables automated generation of both frontend and backend artifacts from a high-level schema. For the backend generator component, the user requirement is to reduce manual backend coding effort by accepting a JSON Schema and automatically producing backend-ready outputs. Specifically, the component must generate DTOs with validation annotations and an OpenAPI specification that represents the defined entities. This satisfies the need for consistency between different layers of the application, ensuring that frontend and backend modules operate on the same validation logic [3][6][7].

4.1.2 System Requirements

At the system level, the backend generator must perform schema processing through a well-defined pipeline. First, the JSON Schema is ingested and verified by a Lexer and Parser, which perform both syntactic and semantic checks. This step ensures that all constraints such as required, minLength, maxLength, minimum, maximum, and pattern are properly captured. Next, an Intermediate Representation is built, which acts as the central model. From this representation, the system generates:

DTOs in either Spring Boot (using Bean Validation annotations such as @NotNull, @Size, @Min).

OpenAPI specifications under components.schemas, ensuring interoperability across services and toolchains such as Swagger Codegen and OpenAPI Generator [5][6][7].

By producing these outputs, the backend generator provides a reliable foundation that can be integrated seamlessly with other system modules, including frontend generators and middleware services [12][13].

4.2 Non-Functional Requirements

4.2.1 Reliability

The component must ensure schema-to-code consistency, so that all generated DTOs and specifications strictly follow the JSON Schema input. Studies on consistency across layers highlight that mismatched rules between frontend and backend lead to failures in data integrity and reliability [3].

4.2.2 Performance

The generator should be efficient, producing DTOs and specifications within seconds for typical schemas. Prior research on automatic specification generation, such as RAAG and transformation-based approaches, emphasizes the need for automation tools to operate at scale without becoming bottlenecks [1][2].

4.2.3 Availability

The backend generator should operate as a standalone service deployable on development environments or integrated pipelines. Middleware studies such as R2SMA demonstrate the value of modular, service-based components that can be accessed reliably without requiring full system redeployment [12].

4.3 Software Requirements

The backend generator requires a stable toolchain to support both generation and integration tasks:

- Validation Libraries: Hibernate Validator for Java Bean Validation.
- Specification Libraries: OpenAPI tooling, such as Swagger Codegen or OpenAPI Generator [5][6][7].
- Environment: Docker for modular deployment, IDEs such as IntelliJ or Visual Studio, and GitHub for version control and CI.

These requirements are aligned with existing backend automation practices that emphasize flexibility and portability across environments [1][6][13].

5. DESCRIPTION OF PERSONAL AND FACILITIES

5.1 Gantt Chart

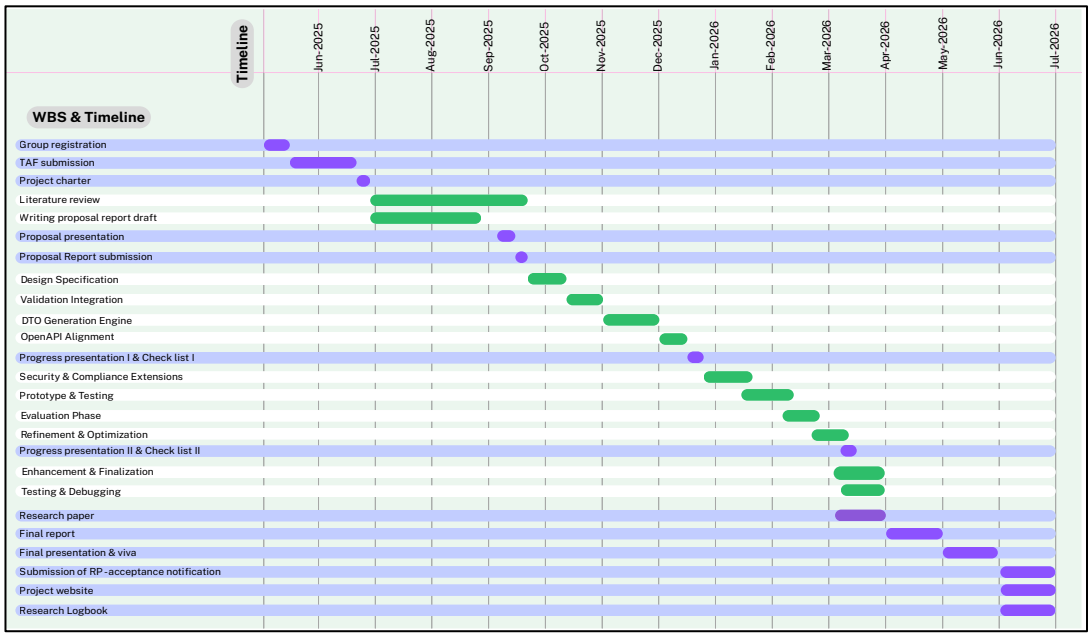


Figure 5.1 - Gantt Chart for the Timeline of the project

5.2 Work Breakdown Chart

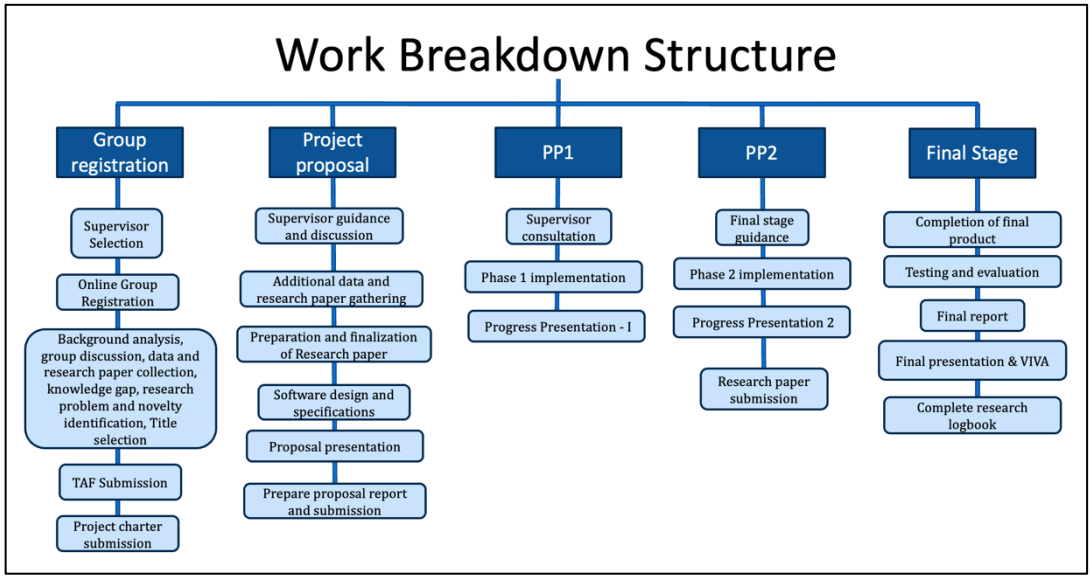


Figure 5.2 - Work Breakdown Structure

6. COMMERCIALIZATION PLAN

The commercialization plan outlines how the backend generator and security integration framework can extend beyond academic research into practical industry adoption. It emphasizes the relevance of the problem, identifies potential user groups, and defines clear pathways for uptake in real-world development environments.

6.1 Business Plan

6.1.1 Market Review & Target Markets

- Independent developers and small backend teams: Teams frequently face repetitive tasks in DTO creation, validation wiring, and contract synchronization. This tool offers automation to reduce errors and increase productivity.
- Software product companies in regulated domains (finance, healthcare, insurance): These sectors must ensure GDPR/HIPAA compliance and audit-ready validation. The framework reduces compliance burden by embedding regulatory checks directly into the backend code.
- Digital agencies and enterprise IT service providers: Agencies focused on rapid prototyping and short delivery cycles can use automated backend generation to minimize mismatches between frontend and backend layers while ensuring security consistency.

6.1.2 Value Proposition

- Time Efficiency: DTO generation, validation annotations, and OpenAPI synchronization reduce backend preparation time by an estimated 40%–60%.
- Security Compliance: Automated enforcement of OWASP-aligned security validations (regex sanitization, input constraints) and compliance tagging for PII ensure regulatory adherence with reduced manual effort.
- Consistency: Eliminates validation drift between frontend and backend through schema-driven generation and CI-based consistency checks.

- Auditability: GDPR-tagged metadata and automated compliance reporting provide traceability for organizations subject to audits.

6.1.3 Business Model Approach

- Freemium SaaS Model
  - Free Tier: DTO and OpenAPI generation for individuals, open-source contributors, and academic use.
  - Professional Tier: Collaboration support, CI/CD integration, and advanced validation rule support (cross-field, conditional).
  - Enterprise Tier: Includes compliance modules (GDPR/HIPAA), API access, dedicated support, and integration with enterprise security workflows.

6.1.4 Cost and Resource Structure

- Development: Achieved during the research project phase (prototype implementation, testing, and refinement).
- Operation: Cloud hosting, schema repository storage, and validation service maintenance.
- Support: Documentation, tutorials, developer forum, and enterprise support desk.
- Marketing: Academic publications, developer community outreach, technical webinars, and case study showcases.

6.1.5 Commercial Viability Factors

- Technical Scalability: Cloud-native deployment with ability to scale backend validation services for enterprise-grade workloads.
- Market Interest: Developer surveys indicate a strong demand for backend automation tools with compliance features.

- Competitive Edge: Existing schema generators lack integrated compliance enforcement and security validation, giving this framework a distinct advantage.

6.2 Marketing Plan

- Target Customers: Enterprise software teams (finance, healthcare, e-government), digital transformation consultants, and developer-focused SaaS platforms.
- Channels: GitHub marketplace plugins, developer forums, industry conferences (e.g., API World, DevSecCon), university research partnerships.
- Promotional Activities: Free 30-day enterprise trial, webinars on secure backend generation, compliance whitepapers, and open-source reference projects.
- Metrics: Trial-to-paid conversion rate, CI/CD adoption statistics, enterprise license sales, and customer retention rate.

7. BUDGET AND BUDGET JUSTIFICATION

Category	Item	Cost (LKR)	Justification
Internet & Connectivity	University WiFi Access	0	Campus high-speed connectivity covers development phase.
Software & Services	Static Analysis & Vulnerability Scanners (SonarQube, OWASP ZAP)	20,000	Required for backend security validation.
Cloud Deployment	AWS/Azure/GCP Extended Credits	15,000	Hosting of schema repository and validation services.
	Premium Development Tools (JetBrains, IntelliJ)	5,000	Efficient Spring Boot backend generation.
Database Hosting	PostgreSQL / MongoDB	4,500	Managed backend for generated DTO and schema storage.
Load Balancer & CDN	Cloud Load Balancer, CDN	3,000	Ensures reliable delivery of validation services.
Data & Resources	Research Paper Access (IEEE/ACM)	0	Covered via university library.
Testing & Validation	CI/CD Pipeline Testing Tools (GitHub Actions, Jenkins)	2,500	Automated validation and consistency checking.
	User Testing Platform (Premium)	1,500	Collects developer usability feedback.
Contingency	Emergency Fund	5,300	Covers unforeseen expenses.
TOTAL PROJECT COST		**58,800**	Comprehensive backend-focused implementation.

Table 7.1 - Budget Table

9. REFERENCES

[1] J. Dazine, A. Maizate, and L. Hassouni, *Weberator: A Low Code Backend Generator Tool*, 2023.

[2] V. Samotyy, U. Dzelendzyak, and N. Mashtaler, "A Comparative Study of Data Annotations and Fluent Validation in .NET," *International Journal of Computing*, vol. 23, no. 1, pp. 72–77, Mar. 2024, doi: 10.47839/ijc.23.1.3437.

[3] H. Park, J. Hwang, and S. Kim, "LRASGen: LLM-based RESTful API Specification Generation," in *Proc. ACM Int. Conf. on Software Engineering and Knowledge Engineering*, 2025.

[4] E. Swathi and A. R. Vanga, "Transformers based Python Code Generation from Natural Language," in *Proc. Int. Conf. on Innovative Trends in Information Technology (ICITII-T)*, 2024, doi: 10.1109/ICITII-T61487.2024.10580762.

[5] F. Tanveer, F. Iradat, W. Iqbal, and A. Ahmad, "Towards Secure APIs: A Survey on RESTful API Vulnerability Detection," *Computers, Materials & Continua*, vol. 77, no. 2, pp. 345–370, Jul. 2025, doi: 10.32604/cmc.2025.067536.

[6] D. Basin, M. Clavel, and J. Doser, "A Decade of Model-Driven Security: Concepts, Analysis, and Applications," in *Proc. ACM Symp. Access Control Models and Technologies (SACMAT)*, pp. 1–10, 2011, doi: 10.1145/1998441.1998443.

[7] D. Basin, F. Klaedtke, S. Müller, and E. Zălinescu, "A Model-Driven Methodology for Developing Secure Data-Management Applications," *IEEE Trans. Software Engineering*, vol. 40, no. 4, pp. 368–383, Apr. 2014, doi: 10.1109/TSE.2013.2297119.

[8] A. Golmohammadi, A. Mesbah, and K. Pattabiraman, "Testing RESTful APIs: A Survey," *ACM Computing Surveys*, vol. 56, no. 3, pp. 1–41, Jun. 2023, doi: 10.1145/3583558.

[9] D. Corradini, M. Pasqua, and M. Ceccato, "Automated Black-box Testing of Mass Assignment Vulnerabilities in RESTful APIs," in *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, pp. 1–13, 2023, doi: 10.1109/ICSE.2023.00123.

[10] A. Mazidi, D. Corradini, and M. Ghafari, "Mining REST APIs for Potential Mass Assignment Vulnerabilities," in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2024, doi: 10.1145/nnnnnnn.nnnnnnn.

[11] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, M. Zhou, and B. Qin, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, pp. 1536–1547, 2020.

[12] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “CodeT5: Identifier-Aware Unified Pre-trained Encoder–Decoder Models for Code Understanding and Generation,” in *Proc. EMNLP*, pp. 8696–8708, 2021.

[13] M. Thüm, D. Koss, I. Schaefer, and G. Saake, “Yo Variability! JHipster: A Playground for Analyses of Web Applications,” in *Proc. ACM Int. Systems and Software Product Line Conf.*, pp. 1–8, 2017.

[14] E. Lopez-Herrejon, R. Rabiser, and A. Egyed, “Assessing Configuration Sampling on the JHipster Web Development Stack,” in *Proc. Int. Conf. on Software Product Line (SPLC)*, pp. 286–296, 2018.

[15] P. H. Nguyen, M. Kramer, J. Klein, and Y. Le Traon, “A Systematic Review on Model-Driven Development of Secure Systems,” *Information and Software Technology*, vol. 55, no. 8, pp. 1473–1499, 2013, doi: 10.1016/j.infsof.2013.03.006.